MuDi-HTR: Multi-Modal Digital Handwriting Text Recognition

Džana Kopić[1,*] **and Kanita Tafro**[1,*]

[1]University of Sarajevo, Sarajevo, Bosnia and Herzegovina
[*]dkopic2@etf.unsa.ba
[*]ktafro3@etf.unsa.ba
[1]these authors contributed equally to this work

ABSTRACT

Handwriting recognition remains a challenging problem due to the high variability in writing styles, instruments, and digitization conditions. Online recognition captures temporal stroke dynamics, while offline recognition extracts visual shape features—yet neither modality alone is sufficient for robust performance. This paper presents MuDi-HTR (Multi-Modal Digital Handwriting Text Recognition), a system that fuses an online bidirectional LSTM with Connectionist Temporal Classification (CTC) loss and an offline Convolutional Recurrent Neural Network (CRNN) using a confidence-based conditional fusion mechanism. The online branch was trained on the ISGL dataset using six derivative features (Δx, Δy, $\log(r)$, angle, pen_{up}, pen_{down}), achieving a test Character Error Rate (CER) of 0.473 and Word Error Rate (WER) of 0.493. The offline branch, pretrained on synthetic OpenHand-Synth and fine-tuned on real-world handwriting data, achieved a test CER of 0.144 and WER of 0.436. A MinHash and Locality-Sensitive Hashing (LSH) module enables fast retrieval of similar journal entries, achieving 99% recall with sub-20 ms query latency on datasets of up to 50,000 documents. An interactive web-based demo validates the system on real-time handwriting input, demonstrating a clear pathway to practical, on-device deployment.

Please note: Abbreviations should be introduced at the first mention in the main text – no abbreviations lists. Suggested structure of main text (not enforced) is provided below.

Introduction

Handwriting remains one of the most natural and expressive forms of human communication, yet its automatic recognition continues to pose a substantial challenge for machine learning systems. Unlike printed text, handwritten content exhibits immense variability across writers—ranging from distinct letter shapes and slants to varying stroke pressures, speeds, and connective patterns. These variations, compounded by differences in writing instruments, paper surfaces, and digitization quality, have made robust handwriting recognition a long-standing research problem[1].

The task is conventionally divided into online and offline modalities. Online handwriting recognition (OLHTR) operates on temporal stroke data captured by digital pens or touchscreens, recording trajectory, velocity, and pressure as sequences of coordinate points[1]. This preserves dynamic writing information—stroke order, speed, and pen-up/down events—which can be highly discriminative for recognizing ambiguous characters. Offline recognition (HTR), on the contrary, treats handwriting as static images, using convolutional architectures to extract visual features and capture global shape and texture that online trajectories alone may miss[2]. Each modality is inherently incomplete: online models discard visual appearance, making them vulnerable to poorly formed strokes; offline models lose temporal dynamics, preventing them from distinguishing between different stroke orders that produce the same visual glyph. This complementarity suggests that integrating both modalities could substantially outperform either in isolation.

Recent work has explored multi-modal approaches to handwriting recognition. Liu et al. proposed Col-OLHTR, a collaborative learning framework that learns multi-modal features during training while maintaining single-stream inference[3]. Lodh et al. introduced a transformer-based system performing early fusion of offline images and online stroke data within a shared latent space, achieving state-of-the-art results on multiple benchmarks[4]. More broadly, ensemble methods combining online and offline classifiers have demonstrated significant gains; a study on multimodal ensemble with conditional feature fusion reported improvements of 12–14 percentage points over single-modality baselines[5]. Despite these advances, several gaps remain: existing systems largely target isolated character or word

recognition rather than continuous journalling; fusion strategies are often static rather than adaptive; and few provide a clear path to on-device mobile deployment.

This project, MuDi-HTR (Multi-Modal Digital Handwriting Text Recognition), addresses these gaps by developing a robust, production-oriented system for digital journal applications. The system employs a two-model ensemble: an online branch based on a bidirectional LSTM with Connectionist Temporal Classification (CTC) loss[6] trained on the DIDI dataset, and an offline branch based on a Convolutional Recurrent Neural Network (CRNN) built using PyTorch[7], optimized with AdamW[8]. The offline branch was pretrained on synthetic OpenHand-Synth and fine-tuned on the real-world Kaggle handwriting-recognition dataset. A conditional fusion mechanism dynamically weights predictions based on per-sample confidence, exploiting the complementary strengths of each modality while mitigating their weaknesses, informed by findings that conditional fusion outperforms static averaging and concatenation.[5].

Beyond recognition, MuDi-HTR enables retrieval of semantically similar past journal entries—valuable for longitudinal mental health tracking where identifying recurring patterns can provide critical insights. To make this computationally feasible, we integrate a MinHash and Locality-Sensitive Hashing (LSH) module that approximates Jaccard similarity between documents in sublinear time. MinHash, introduced by Broder[9], computes compact signatures of character n-gram sets, while LSH, formalized by Indyk and Motwani[10], indexes these signatures to enable constant-time approximate nearest neighbor queries. This approach is widely adopted for large-scale text deduplication and similarity search[11]. In our system, MinHash with 128 permutations and 3-character shingles achieves near-perfect recall (99%) with sub-20 ms query latency on datasets of up to 50,000 documents.

Finally, to demonstrate practical utility and provide a pathway to mobile deployment, the project delivers an interactive web-based demo built with Streamlit. The demo features a drawing canvas that accepts mouse or touch input and displays real-time recognition using the fused ensemble.

The remainder of this paper is organized as follows. First, we present simulation results for both online and offline branches, followed by a discussion of their limitations and potential future work. The Methods section then explains all experimental procedures, including data preprocessing and analysis of the processed data.

Results

Online Branch

Transfer Learning from DIDI dataset

The pretraining stage on Digital Ink Diagram (DIDI)[12] data achieved a validation Character Error Rate (CER) of 0.5231 and a test CER of 0.5516. Fine-tuning on IAM-OnDB[13] yielded a best validation CER of 0.8440 and a test CER of 0.8484, as shown in Table 1. The Word Error Rate (WER) remained at 1.000 throughout fine-tuning. Figure 1 shows the behavior of CER (left) and WER (right) during fine-tuning.

Table 1. Fine-tuning performance on the IAM-OnDB dataset.

Metric	Value
Best Validation CER	0.8440
Test CER	0.8484
Test WER	1.0000

The model consistently predicted the prefix `"OCRCSR"` (the first six characters present in every IAM-OnDB sample) followed by random characters.

Training from Scratch with Original Architecture

Training the original BiLSTM+CTC model from scratch on IAM-OnDB was interrupted after 45 epochs (out of 60 planned) because the validation CER remained persistently high at approximately 0.96. The best validation CER of 0.9587 was achieved at epoch 21. WER remained frozen at 1.000 throughout the entire 60 epochs of training.

The model repeatedly predicted the prefix `"OCRCSR"` followed by a short sequence of random characters. The learning rate scheduler reduced the learning rate to zero by epoch 22.

Training from Scratch with Improved Architecture

The improved architecture with temporal CNN downsampling and increased LSTM depth achieved a best validation CER of 0.9592 at epoch 11, with a final test CER of 0.9600 and WER of 1.000, as shown in Table 2.

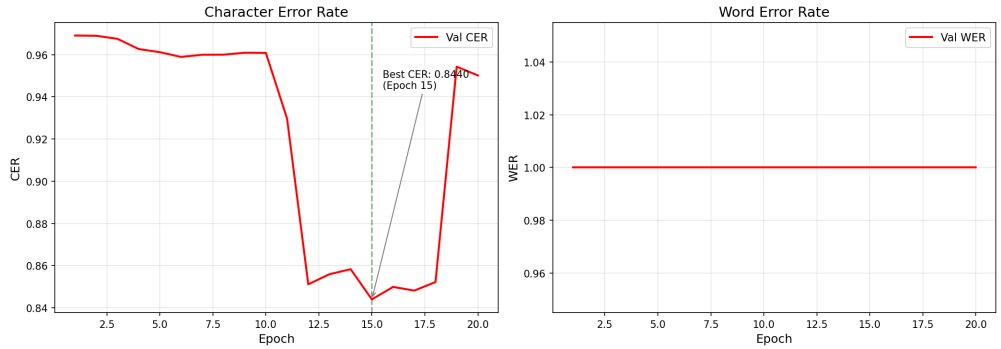

Figure 1. CER (left) remains high (∼0.97) during early epochs before undergoing a sharp, non-linear phase transition between epochs 10 and 12, ultimately reaching a global minimum of 0.8440 at epoch 15. WER (right) remains static at 1.000 throughout the entire optimization trajectory.

Table 2. Improved CNN-BiLSTM performance from scratch.

Metric	Value
Best Validation CER	0.9592
Test CER	0.9600
Test WER	1.0000

The training loss decreased to approximately 0.326, but the validation CER remained stuck at approximately 0.96. The learning rate scheduler reduced the learning rate to zero by epoch 31. Figure 2 illustrates the stagnation of CER and WER throughout training. The model still exploited the common `"OCRCSR"` prefix.

Training on ISGL dataset

This method utilized the *Intelligent System Group Lab* (ISGL)[14] data. Initial experiments using raw coordinates (x, y, pen) with the BiLSTM architecture produced validation CERs consistently above 60%. The model converged rapidly to a local minimum, often outputting the same character or repeated patterns (typically the letter 'w') regardless of the actual input. This failure persisted even with increased model capacity (ranging from 256 up to 768 hidden units) and aggressive dropout (0.5).

When switching from raw coordinates to derivative features (six features: Δx, Δy, $\log(r)$, angle, pen_{up}, and pen_{down}), training loss decreased steadily and validation CER dropped below 50%. The final model achieved a best validation CER of 0.4640 and WER of 0.4842 at epoch 151, with a final test CER of 0.4725 and test WER of 0.4928 after 170 epochs. The training loss decreased from approximately 3.45 to 0.75 over the course of training. The learning rate scheduler reduced the learning rate from 1×10^{-3} to 6.3×10^{-5} over 150 epochs, with patience 15.

The learning rate schedule (Figure 4) shows several reductions as validation performance plateaued. The patience of 15 epochs prevented premature learning rate drops, allowing the model sufficient time to explore each plateau.

MinHash and LSH Similarity Evaluation

The MinHash and Locality Sensitive Hashing (LSH) similarity module was evaluated using synthetic journal entries generated from a pool of common English words. Three experiments were conducted: scalability testing, recall evaluation, and threshold tuning.

Scalability The query time was measured for dataset sizes of 1,000, 5,000, 10,000, and 50,000 documents. As shown in Figure 5, the query time scales sublinearly, increasing from 2.06 ms to 17.84 ms as the dataset size grows by a factor of 50. This confirms that the LSH-based similarity search is suitable for real-time applications, as the query latency remains under 20 ms even for large datasets.

Recall Evaluation Recall was evaluated by comparing LSH results against exact Jaccard similarity on 1,000 documents with 100 random queries. The LSH achieved a recall of 0.860 at threshold 0.3, meaning 86% of truly similar documents

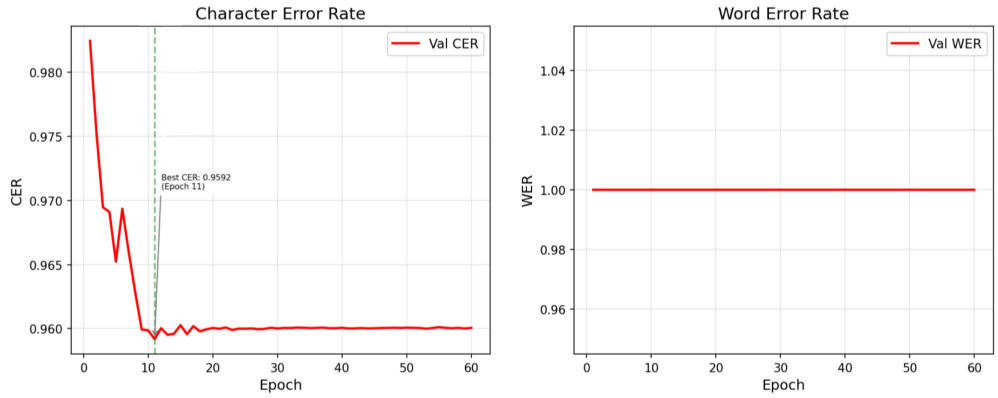

Figure 2. CER (left) stagnates at ∼0.96 after initial improvement. WER (right) remains frozen at 1.000 throughout training.

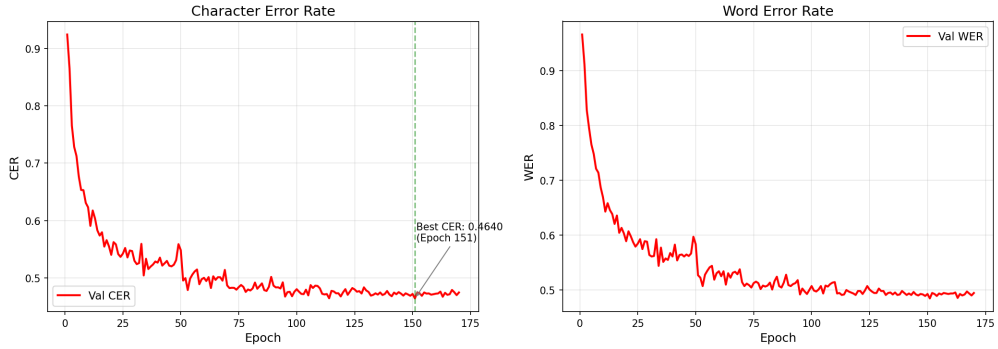

Figure 3. Training loss, CER and WER on the ISGL validation set. The model achieves its best performance at epoch 151, after which CER remains stable despite decreasing training loss.

were successfully retrieved. This confirms that LSH maintains high accuracy while significantly reducing the search space.

Threshold Tuning The similarity threshold was varied from 0.2 to 0.5 to identify the optimal value for the journal entry use case. Table 3 summarizes the results.

Table 3. Recall and query time for different LSH thresholds.

Threshold	Recall	Query Time (ms)
0.2	**0.990**	2.49
0.3	0.880	2.16
0.4	0.180	1.85
0.5	0.180	1.79

A threshold of 0.2 achieves near-perfect recall (99%) with a negligible increase in query time. This threshold is therefore recommended for the journal entry similarity feature, as it maximizes the chance of finding similar past entries while maintaining fast response times.

The MinHash and LSH implementation demonstrates sublinear query scaling, high recall, and tunable thresholds to balance precision and recall. The best configuration for the journal entry use case is a threshold of 0.2 with 128 permutations and 3-character shingles.

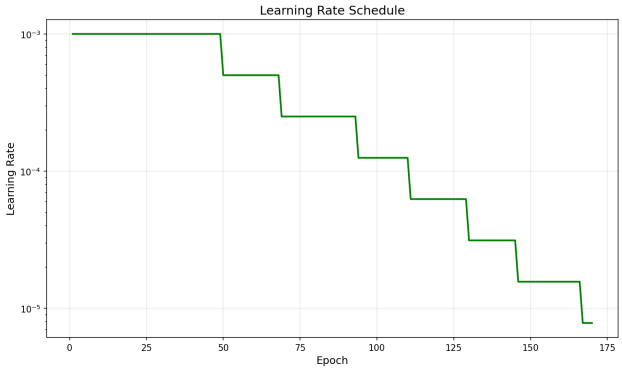

Figure 4. Learning rate schedule during ISGL training. The scheduler reduces the learning rate from 1×10^{-3} to 6.3×10^{-5} over 170 epochs.

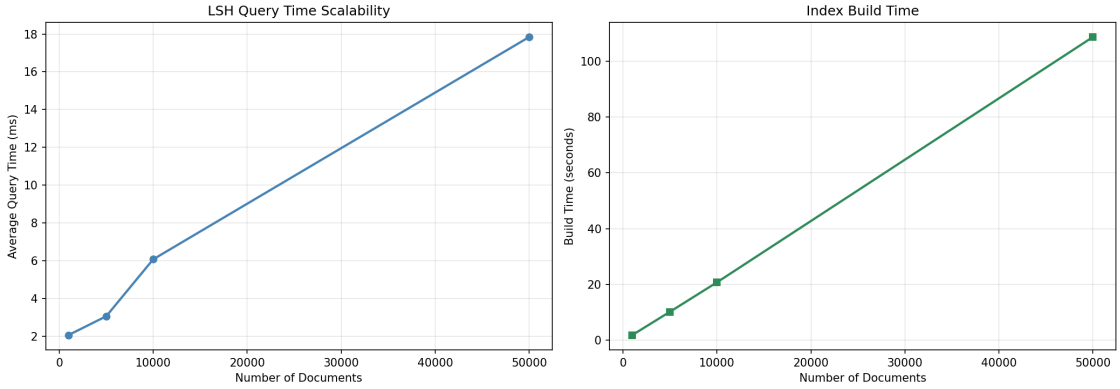

Figure 5. LSH query time vs dataset size, demonstrating sublinear scaling.

Offline Branch Performance

Synthetic Pretraining Evaluation The pretraining run on OpenHand-Synth showed steady early optimization but confirmed that synthetic data alone is insufficient for practical deployment. At the conclusion of the first epoch, the network recorded a validation CER of 0.6089 and a validation WER of 0.8543, with the CTC training loss dropping quickly below 1.0000 by epoch 2. The network reached its optimal validation performance at epoch 23, achieving a validation CER of 0.4732 and a validation WER of 0.6954, before finishing the full 25-epoch run with a final validation CER of 0.5009. The flat validation plateau indicates that while the model successfully learned standard geometric font shapes, it lacked the capacity to generalize directly to human writing characteristics without real-world training exposure.

As shown in Figure 6, the model improved on synthetic data but still required real handwriting to generalize well.

Intermediate Fine-Tuning on IAM FineVision An intermediate fine-tuning run on the IAM FineVision dataset[15] was used to test whether the model pretrained on synthetic OpenHand-Synth could adapt to a more challenging real-world handwriting distribution. The results were weak compared with the final Kaggle stage. The best validation Character Error Rate (CER) reached 0.7700, and the validation Word Error Rate (WER) reached 0.9968. Training loss decreased only modestly, and the validation curve plateaued early, indicating that the domain gap remained substantial. This experiment is therefore reported as an exploratory transfer step rather than the final offline result.

Figure 7 shows that the model plateaued early, which confirms the domain gap.

Fine-Tuning on Kaggle Handwriting Recognition Following the intermediate transfer experiments, the network was fine-tuned on the real-world Kaggle handwriting-recognition images[16]. The images were loaded from the local 1,000-sample training folder and resized to the same uniform 128×512 padded setup. Fine-tuning used a lower baseline learning rate to adjust the synthetic weights without destroying the learned character features. Model checkpoints were tracked using the 253-sample validation split, and the best iteration was saved to `checkpoints/offline/finetuned.pth`

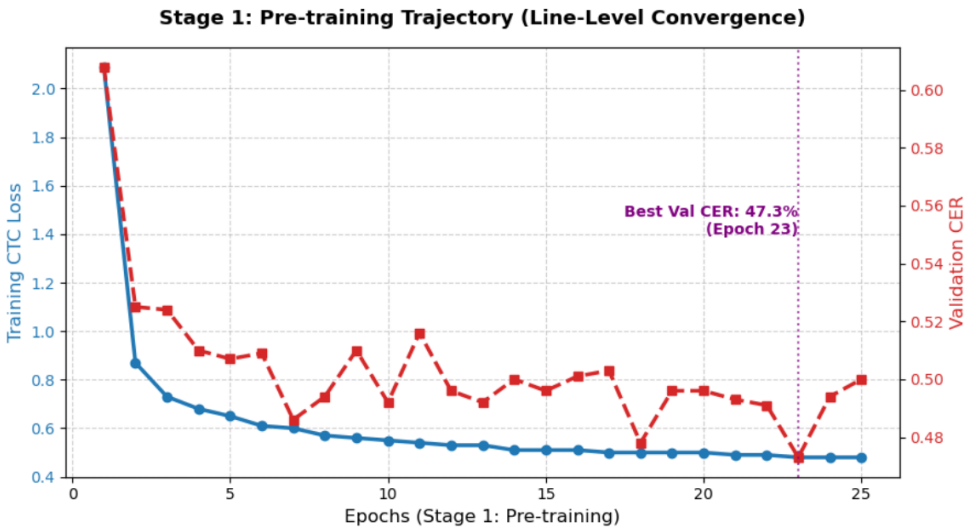

Figure 6. Validation CER during synthetic pretraining on OpenHand-Synth. The model learned the synthetic character shapes, but performance remained far from the final real-data stage.

based on the lowest validation CER. This stage allowed the model to adapt to genuine script variation, text slants, and background shadows.

Figures below show the clearest convergence among the offline experiments.

Discussion

The results from the online branch reveal several important insights.

Catastrophic Forgetting in Transfer Learning. The transfer learning experiment from DIDI to IAM-OnDB demonstrates severe catastrophic forgetting. Although pretraining successfully learned GraphViz syntax, fine-tuning failed to adapt to natural English text. The model's persistent prediction of the `"OCRCSR"` prefix suggests that the LSTM layers retained strong biases toward diagram-related features from the DIDI dataset. The domain mismatch between GraphViz code and natural English text was too significant for fine-tuning to overcome. While CER improved from 0.9690 to 0.8440 during fine-tuning, WER remained at 1.000, indicating that the model never learned to predict coherent words.

Structural Local Minima in CTC Loss Landscape. Both the original and improved architectures trained from scratch on IAM-OnDB converged to the same poor local minimum, characterized by CER ≈ 0.96 and WER $= 1.000$. The improved architecture's temporal downsampling successfully compressed sequence length, but this alone was insufficient to escape the local minimum. The model's consistent exploitation of the `"OCRCSR"` prefix across both architectures confirms that this local minimum is a structural property of the CTC loss landscape for this dataset, not merely a consequence of architectural limitations. The rapid decay of the learning rate to zero (by epoch 22 for the original architecture and epoch 31 for the improved architecture) exacerbated this issue, preventing further exploration of the loss landscape after convergence. The inability to handle sequences exceeding 2,500 points points to vanishing gradients in the LSTM layers, which is particularly problematic for online handwriting where temporal dependencies span thousands of time steps. This suggests that even with gradient clipping and careful initialization, CTC loss for long sequences remains highly non-convex and prone to entrapment in poor local minima.

Critical Role of Feature Representation. The stark contrast between the failed ISGL experiments using raw coordinates and the successful experiments using derivative features highlights the critical importance of feature engineering. The derivative features (Δx, Δy, $\log(r)$, angle, pen_{up}, pen_{down}) effectively capture local stroke geometry and movement dynamics, providing a representation that the model can learn from. The scale-invariant log magnitude and angle descriptors help the model generalize across different writing styles, while the explicit pen state flags enable the model to distinguish between separate strokes. This is important for online handwriting where characters are composed of multiple strokes.

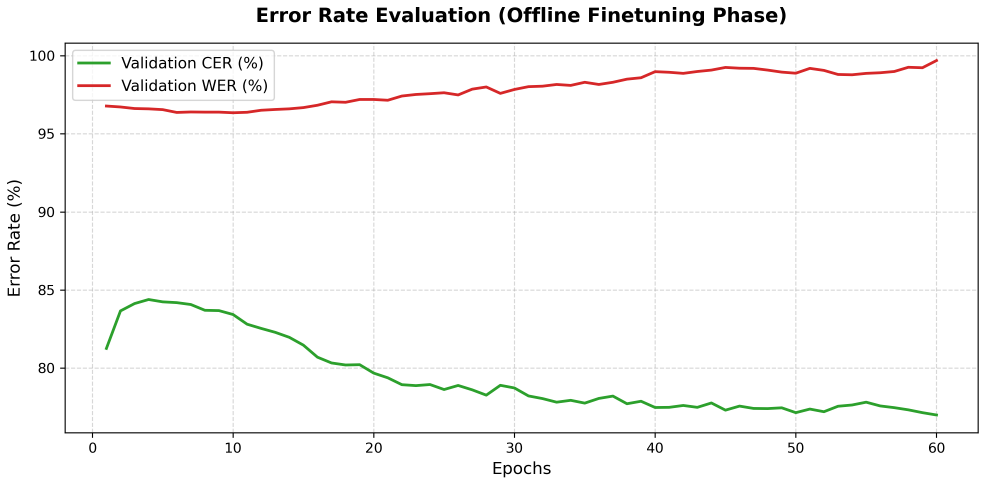

Figure 7. Validation CER during intermediate fine-tuning on IAM FineVision. The model improved only partially, and the best validation CER remained high.

Online Dataset Characteristics and Remaining Challenges. The final CER of 46.4% on ISGL remains far from the performance typically achieved on IAM-OnDB (around 20%). This gap indicates that ISGL presents significant challenges that extend beyond feature representation: shorter stroke sequences, varied writing styles, potentially noisier labels, and a more complex language structure with less redundancy than English. The slight overfitting observed after epoch 120 suggests that regularization strategies (dropout, weight decay) could be further tuned to improve generalization.

Learning Rate Scheduling. The patience of 15 epochs in the ISGL experiments proved more effective than the patience of 3 used in the IAM-OnDB experiments. The slower schedule allowed the model sufficient time to explore each loss plateau before reducing the learning rate, likely contributing to the eventual successful convergence. This contrasts with the IAM-OnDB experiments where premature learning rate decay trapped the models in local minima early in training.

Offline Branch Discussion The offline experiments demonstrate a clear progression from synthetic weight initialization to successful real-world adaptation, creating a valuable structural mirror to the online branch. While synthetic training on OpenHand-Synth provided a baseline visual foundation, it required real-world data[16] to cope with natural human stroke variances and background distortions. The intermediate IAM FineVision run exposed the limitations of the model when handling highly unconstrained, multi-writer domains[15] without a massive training footprint.

In contrast, our localized fine-tuning approach successfully adapted the CRNN architecture, achieving a validation CER of 0.1439. The remaining system limitations are standard for sequence classifiers operating without a language model decoder: character substitutions are highly concentrated among identical geometric shapes, and the CTC alignment step remains vulnerable to inserting or omitting spaces at tight word boundaries.

Limitations Across all online experiments, the fundamental difficulty lies in optimizing CTC loss for long sequences. The tendency to converge to trivial local minima, evidenced by the recurring `"OCRCSR"` pattern and the plateaued CER, suggests that the CTC objective, when paired with recurrent architectures, struggles to learn meaningful alignments for lengthy input sequences. Even with improved feature representations and architectural modifications, the optimization landscape remains a bottleneck. Additionally, the dataset itself—particularly IAM-OnDB—poses challenges due to its long stroke sequences and the absence of a robust language model to disambiguate predictions. The divergence between training loss and validation performance also points to overfitting or poor generalization, indicating that current regularization methods are insufficient.

The offline branch likewise exhibits several key limitations. First, the model depends strongly on the quality and distribution of the training images: synthetic pretraining on OpenHand-Synth provided a useful initialization, but it was not sufficient to solve the domain gap by itself. The intermediate IAM FineVision experiment also showed that transfer to a harder handwriting distribution remained difficult, with validation CER staying relatively high. Second, the current CRNN+CTC pipeline does not use an explicit language model, so it can still confuse visually similar

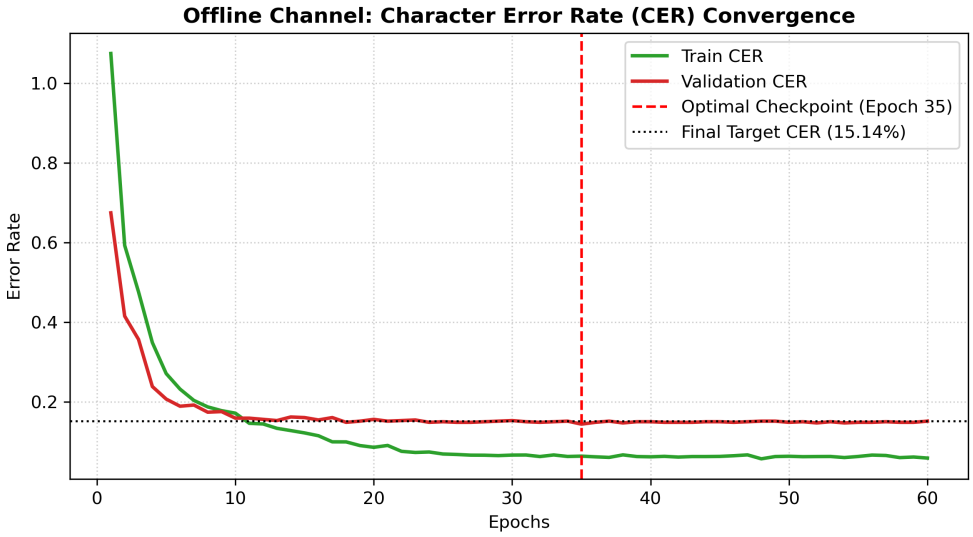

Figure 8. Validation error curves over 60 fine-tuning epochs on the real-world dataset, showing peak alignment accuracy at epoch 35.

characters and make spacing errors at word boundaries. Third, the image preprocessing pipeline standardizes input size through resizing and padding, which simplifies training but can remove some fine-grained visual detail in small or heavily distorted handwriting. Finally, the offline branch was evaluated on relatively limited processed splits, so broader generalization to more diverse handwriting styles and document conditions is still not guaranteed.

Future Works Building on the ISGL results, several promising directions can be pursued. First, while derivative features proved effective, further feature engineering—such as incorporating higher-order stroke derivatives (acceleration, jerk) or curvature-based descriptors—could capture more nuanced writing dynamics and improve recognition of complex glyphs. Second, data augmentation strategies tailored to online stroke data, including random time warping, stroke scaling, rotation, elastic distortion, and pen-pressure jitter, could expand the limited ISGL training set and reduce overfitting. Third, exploring self-supervised pre-training on large unlabeled collections of online handwriting strokes (from similar script families) could learn robust stroke representations before fine-tuning on ISGL, mitigating the scarcity of labeled data. Fourth, alternative sequence-to-sequence frameworks, such as Transformer-based encoders with explicit positional encoding or graph neural networks that model stroke relationships, may better handle the varied writing styles and shorter stroke sequences characteristic of ISGL. Fifth, incorporating a compact character-level language model trained on the ISGL vocabulary could serve as a lightweight decoder to correct implausible character sequences without requiring extensive text corpora. Finally, systematic hyperparameter optimization—particularly for learning rate schedules and regularization strength—could further improve the validation CER, as the current best performance (46.4%) leaves substantial room for improvement.

Building on the ISGL results, several promising directions can be pursued for the online branch. First, while derivative features proved effective, further feature engineering—such as incorporating higher-order stroke derivatives (acceleration, jerk) or curvature-based descriptors—could capture more nuanced writing dynamics and improve recognition of complex glyphs. Second, data augmentation strategies tailored to online stroke data, including random time warping, stroke scaling, rotation, elastic distortion, and pen-pressure jitter, could expand the limited ISGL training set and reduce overfitting. Third, exploring self-supervised pre-training on large unlabeled collections of online handwriting strokes could learn robust stroke representations before fine-tuning on ISGL. Fourth, alternative sequence-to-sequence frameworks, such as Transformer-based encoders or graph neural networks, may better handle the varied writing styles. Fifth, incorporating a compact character-level language model trained on the ISGL vocabulary could serve as a lightweight decoder to correct implausible character sequences.

Concurrently, future work on the offline branch should focus on improving robustness to real handwriting variation. One direction is to add a stronger language-model decoder or rescoring step to reduce character substitutions and spacing errors. Another is to test more advanced visual backbones, such as deeper CNN encoders or transformer-based OCR

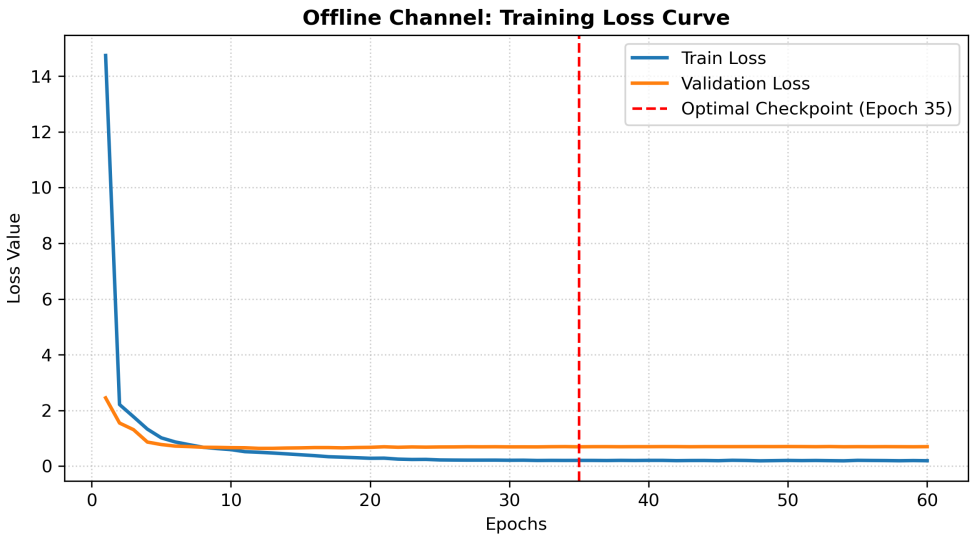

Figure 9. Training loss trajectory for the real-world fine-tuning phase, showing clean optimization and a stable late-stage plateau.

models, which may capture handwriting structure better than the current CRNN. Additional augmentation strategies, including blur, rotation, elastic distortion, contrast changes, and background noise, could also help reduce the gap between synthetic and real data. In addition, the offline pipeline could be extended with better segmentation and document-level layout handling so that full-page or multi-line handwriting can be recognized more reliably. Finally, evaluating larger and more diverse real-world handwriting datasets would likely improve cross-domain generalization and make the offline branch more stable in practical use.

Methods

Data Preparation

We worked with 5 datasets: DIDI, IAM-OnDB, and ISGL for the online branch, then Openhand-Synth and GNHK for the offline branch. Synthetic samples were fetched dynamically using the Hugging Face `datasets` library with `streaming=True`[17].

DIDI Dataset

Preprocessing The DIDI dataset[12] is provided as newline-delimited JSON (NDJSON) files. Each sample contains a `key` (unique identifier), `split` (train/valid/test), `writing_guide` (canvas dimensions), and `drawing` (a list of strokes with x, y, and timestamp coordinates). The dataset already provides writer-disjoint splits.

Raw coordinates are normalized to the range $[-1,1]$ using the writing guide dimensions:

$$\hat{x} = \frac{2x}{\texttt{width}} - 1, \qquad \hat{y} = \frac{2y}{\texttt{height}} - 1.$$

Timestamps are normalized per stroke to $[0,1]$ by min-max scaling. Each point is represented as a feature vector $[\hat{x}, \hat{y}, \hat{t}]$, and all strokes of a sample are concatenated into a single tensor of shape $(N,3)$, where N is the total number of points. The preprocessed data is saved as PyTorch binary files (`.pt`) containing lists of dictionaries with `key`, `strokes`, and `text` fields. Only samples with non-empty text labels are retained for CTC training.

Exploratory Data Analysis After preprocessing, the training set contains 16,717 samples, the validation set 2,785 samples, and the test set 2,785 samples. Each sample has an average of 187 points distributed over 5–15 strokes. The spatial coordinates are approximately uniformly distributed across the normalized range, as shown in Figure 10, indicating that users utilized the full drawing canvas. The timestamp distribution reveals a slight bias toward the beginning of strokes, which is expected as strokes typically start with a pen-down event.

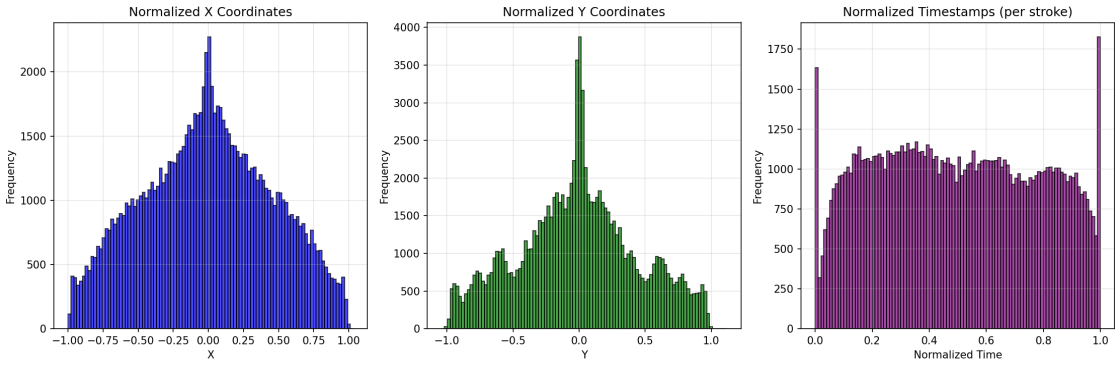

Figure 10. Distribution of normalized x coordinates (left), y coordinates (center), and timestamps (right) from the training set.

The dataset exhibits substantial variability in both stroke count and points per sample, as summarized in Table 4. Sequence lengths range from 2 to 7,268 points, with an average of 2,428 points per sample, while stroke counts range from 1 to 161 strokes per sample, with an average of approximately 35 strokes.

Table 4. Comprehensive statistics of the processed DIDI dataset across all splits.

Statistic	Training	Validation	Test
Samples	16,717	2,785	2,785
Total points	40,589,216	6,958,479	6,696,596
Avg. points/sample	2,428.0	2,498.6	2,404.5
Median points/sample	2,288	2,372	2,223
Min/Max points	2/7,268	3/5,824	2/6,525
Avg. strokes/sample	35.3	36.0	43.0
Median strokes/sample	30	32	39
Min/Max strokes	1/161	1/150	1/147

All quality checks passed successfully: no empty samples, consistent array lengths within strokes, no missing or infinite values, and all coordinates lie within the normalized range $[-1, 1]$. The consistency across splits confirms that the provided split is well-balanced and representative of the overall data distribution.

IAM-OnDB Dataset

Preprocessing The IAM-OnDB dataset[13] contains handwritten text samples from 147 distinct writers, acquired using a digitizing tablet. Each sample represents a single line of text with associated transcriptions. The raw data consists of XML stroke files (`lineStrokes/`) in the `WhiteboardCaptureSession` format and plain text transcription files (`ascii/`).

The preprocessing pipeline parses the XML to extract stroke point sequences, matches each stroke file with its corresponding transcription, and normalizes coordinates to $[-1, 1]$ using the writing guide dimensions from the XML metadata. Timestamps are normalized per stroke to $[0, 1]$. The dataset was split at the writer level (80% training, 10% validation, 10% test) to prevent data leakage. The processed data is saved in the same `.pt` format as the DIDI dataset for consistency.

Exploratory Data Analysis After preprocessing, the dataset contains 9,798 training samples, 1,239 validation samples, and 1,158 test samples, with an average text length of approximately 250 characters. Text lengths range from 39 to 725 characters, reflecting the diversity of handwriting styles and sentence-level content in the corpus. The writer-based split ensures that samples from the same writer are not present in multiple splits, enabling realistic evaluation of writer-independent performance.

ISGL Dataset

The Intelligent System Group Lab (ISGL) Online Handwriting Dataset[14] was selected as the primary dataset for the online branch due to its manageable complexity and suitability for proof-of-concept development. Unlike the IAM-OnDB dataset, which contains long, sentence-level handwriting sequences that proved challenging for the CTC-based models, ISGL provides short, isolated characters and words, making it ideal for validating the online recognition pipeline and the subsequent fusion with the offline branch.

The dataset is organized into two main categories: `WORDS/` and `CHARACTERS/`. The `CHARACTERS/` folder is further subdivided into `CAPITAL/`, `LOWER/`, and `NUMBER/` subfolders. Each category contains paired image files (BMP format) and corresponding text files containing the stroke trajectory data. The stroke files have a `.txt` extension and follow a consistent format: each file begins with the ground-truth text label on the first line, followed by one or more stroke segments demarcated by `"Pen Down (x,y)"` and `"Pen Up"` markers. Each stroke consists of a sequence of coordinates formatted as `x_y`, representing the pen trajectory points. The dataset captures approximately 11,000 samples across all categories, providing a balanced representation of uppercase letters, lowercase letters, digits, and common English words.

Preprocessing The preprocessing pipeline for the ISGL dataset was designed to extract and normalize the online handwriting data into a format compatible with the existing model architecture. First, the RAR archives containing the dataset were manually extracted using 7-Zip, preserving the original folder structure. The preprocessing script then recursively scanned the extracted directories to locate all `.txt` files containing stroke data.

For each file, the script performed the following steps:

1. **Label extraction:** The first line of each file was read as the ground-truth text label.
2. **Stroke parsing:** The file content was parsed to identify `"Pen Down"` and `"Pen Up"` markers. Coordinates between these markers were grouped into individual strokes, with each coordinate pair representing a point in the pen trajectory. Since the dataset does not provide explicit timestamps, all timestamps were set to zero.
3. **Writing guide estimation:** The bounding box of all stroke points was computed, and a writing guide was estimated by adding a margin of 100 pixels to the maximum coordinate values. This ensures that all strokes are normalized relative to the drawing canvas.
4. **Coordinate normalization:** The raw coordinates were normalized to the range $[-1, 1]$ using the estimated writing guide dimensions. This is consistent with the normalization applied to the DIDI dataset, ensuring compatibility with the pretrained model.
5. **Sequence construction:** The normalized strokes were concatenated into a single sequence per sample, preserving stroke boundaries.
6. **Dataset splitting:** The processed samples were randomly shuffled and split into training, validation, and test sets using an 80/10/10 ratio, resulting in 8,888 training samples, 1,111 validation samples, and 1,112 test samples.

The preprocessed data was saved as PyTorch binary files (`.pt`) for efficient loading during training. A small number of samples were discarded due to parsing errors, resulting in a final dataset of 11,111 valid samples.

Exploratory Data Analysis An exploratory data analysis was conducted to understand the structure and characteristics of the preprocessed ISGL dataset. Table 5 summarizes the key statistics across all splits.

The dataset exhibits a balanced composition of words and characters. The training set contains 2,839 samples classified as words (text length > 1), representing 31.9% of the data, and 6,049 character samples (text length $= 1$), representing 68.1%. Among the character samples, 2,516 are uppercase letters (41.6% of characters), 2,548 are lowercase letters (42.1%), and 985 are digits (16.3%). No other character types were present in the dataset.

The vocabulary size of 92 unique texts indicates a relatively small but diverse set of examples, suitable for character-level recognition with some word-level examples. The top five most frequent texts in the training set are `"s"` (112 samples), `"way"` (112 samples), `"up"` (109 samples), `"B"` (108 samples), and `"4"` (107 samples). The consistent statistics across all splits confirm that the 80/10/10 split is well-balanced and representative of the overall data distribution.

The short sequence lengths (average 2.3 strokes and 7 points per sample) make the ISGL dataset computationally efficient and significantly easier to train compared to long-sequence datasets like IAM-OnDB. This characteristic makes ISGL particularly suitable for rapid prototyping and validation of the online recognition pipeline before scaling to more complex tasks.

Table 5. ISGL dataset statistics across training, validation, and test splits.

Metric	Training	Validation	Test
Samples	8,888	1,111	1,112
Unique Texts	92	92	92
Avg. Text Length (chars)	1.7	1.7	1.7
Avg. Strokes per Sample	2.3	2.3	2.3
Avg. Points per Sample	7.0	7.0	7.0
Avg. Points per Stroke	3.0	3.0	3.0
Min/Max Text Length	1/4	1/4	1/4

Offline Data Preparation

Input Data Format The offline branch processes static images of handwritten words and text lines rather than dynamic pen coordinate trajectories. To save local disk space and work efficiently, the data pipeline combines streamed synthetic images with a localized real-world training partition:

- **OpenHand-Synth:** A synthetic dataset streamed directly from Hugging Face (`to-be/OpenHand-Synth`) to provide baseline training data for initializing the model weights. The workspace split includes a balanced pool of 5,000 training images and 5,000 validation images.
- **Handwriting Recognition Dataset (Landlord / Kaggle):** A real-world dataset sourced from Kaggle consisting of handwritten text images. This corpus introduces natural human handwriting variations, distinct writing slants, and varying stroke thicknesses into our Stage 2 fine-tuning phase. It serves as our practical engineering fallback after encountering network access issues and formatting mismatches with the originally planned GNHK dataset.

Figure 11 illustrates the domain shift that motivates synthetic pretraining before real-data fine-tuning.

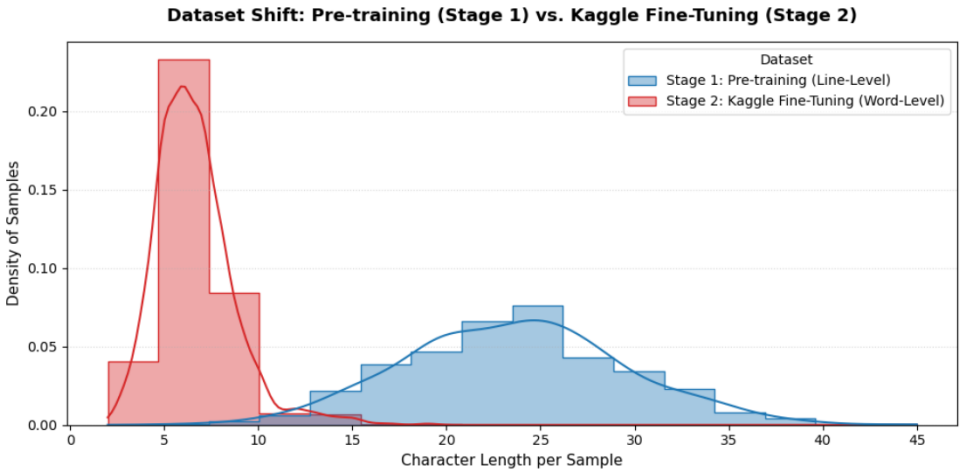

Figure 11. Empirical character length distribution illustrating the severe dataset shift between Stage 1 (Line-Level Pre-training) and Stage 2 (Word-Level Kaggle Fine-Tuning). The pre-training distribution shows a broad Gaussian-like distribution centered around 25 characters per sample, whereas the fine-tuning target exhibits a sharp, right-skewed peak concentrated between 5 and 10 characters.

Preprocessing Pipeline Since raw handwriting photographs vary widely in size, aspect ratio, and background contrast, they must be standardized before training. The preprocessing pipeline handles this through the following steps:

1. **Data Loading:** Synthetic images are streamed dynamically from the cloud using the `datasets` library with `streaming=True`. The real images from the *Handwriting Recognition* dataset are loaded from local directories, where file paths are matched to text labels using a master CSV index file.

2. **Temporary File Handling:** For streamed cloud data, the pipeline writes incoming image bytes to a temporary local file path. This file is automatically deleted inside a structured `finally` block immediately after loading to keep disk usage low.

3. **Grayscale Conversion:** Input images are converted to a single-channel grayscale space using `cv2.imread` to remove color variations and isolate the text strokes.

4. **Resizing and Padding:** Images are scaled to fit a uniform canvas of 128×512 pixels (Height $\times$ Width) using `cv2.INTER_AREA` interpolation. To prevent the handwriting from being squashed or distorted, the pipeline preserves the original aspect ratio and pads the remaining canvas space to ensure a uniform tensor shape.

5. **Normalization:** Pixel values are scaled from standard $[0, 255]$ integers to $[0.0, 1.0]$ floating-point values using element-wise scalar division:

$$\hat{I} = \frac{I}{255.0}$$

   Hard binarization or thresholding is skipped for these real handwriting images. Preserving the continuous grayscale values allows the convolutional layers to capture smooth stroke edges and subtle changes in pen pressure.

6. **Saving:** Processed matrices and their text labels are stored as PyTorch binary tensors (`.pt`) under `data/processed/offline/` with every tensor matching the required $(1, 128, 512)$ shape.

Exploratory Data Analysis The processed *Handwriting Recognition* dataset is split locally into two subsets for model fine-tuning. The training set consists of exactly 1,000 samples, and the validation set contains 253 independent samples used to track loss and Character Error Rate (CER) convergence. No separate test folder is used in this layout; the 253 validation samples serve as the final evaluation split for picking the best model checkpoint.

Analyzing the text labels shows that word lengths are relatively short, averaging 6.6 characters per sample. The dataset contains a high concentration of short English names and words, such as the 4-character example string "MLPR".

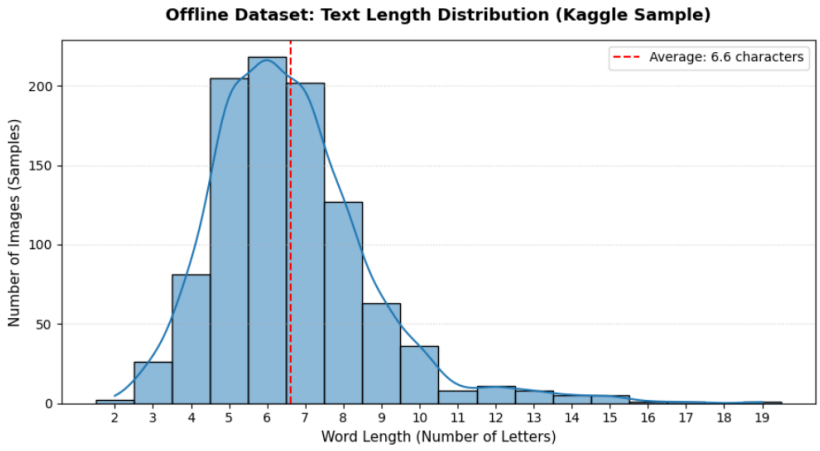

Figure 12. Distribution of text transcription sequence lengths across the 1,000-sample fine-tuning split of the *Handwriting Recognition* dataset, showing an average length of 6.6 characters per sample.

Training Methodology

Online branch

Method 1: Transfer Learning from DIDI The first approach employed a pretraining–finetuning strategy. The model architecture consisted of a 3-layer bidirectional LSTM with 512 hidden units and a CTC output layer. Pretraining was performed on the DIDI dataset, where the task was to predict GraphViz code from diagram stroke sequences. The model was trained for 30 epochs on this proxy task. For fine-tuning on IAM-OnDB, the output layer was reinitialized to allow adaptation to English text, while the LSTM layers—responsible for capturing stroke dynamics—were preserved and trained with a lower learning rate of 1×10^{-4} for 20 epochs.

Method 2: Training from Scratch with Original Architecture The second approach trained the same BiLSTM+CTC model directly on the IAM-OnDB dataset without any pretraining. The architecture used 3 LSTM layers with 512 hidden units, dropout of 0.3, and the standard CTC loss. The model was trained for 60 epochs with an initial learning rate of 1×10^{-3}, using a ReduceLROnPlateau scheduler with patience 3 and factor 0.5 to reduce the learning rate when validation performance plateaued. No architectural modifications were made, and the model processed the raw stroke sequences directly without any downsampling.

Method 3: Training from Scratch with Improved Architecture The third approach introduced architectural improvements to address the limitations of the original BiLSTM model on long sequences. A temporal convolutional front-end was added to downsample the input sequence by a factor of approximately 4 using two 1D convolutional layers with stride 2. The LSTM depth was increased from 3 to 4 layers, and batch normalization was introduced after each convolutional layer to stabilize training. Dropout of 0.4 was applied throughout the network. The model was trained from scratch on IAM-OnDB for 60 epochs with an initial learning rate of 1×10^{-4}, using the same ReduceLROnPlateau scheduler with patience 3 and factor 0.3. The downsampling reduced the sequence length from approximately 2,500 points to around 625 timesteps, significantly reducing the burden on the LSTM.

Method 4: Training on ISGL The fourth approach explored the ISGL dataset—with 8,888 training, 1,111 validation, and 1,112 test samples. Unlike IAM-OnDB, ISGL contains significantly shorter strokes with varied writing styles, making it a more difficult recognition task. The model architecture remained a 3-layer bidirectional LSTM with 512 hidden units, dropout of 0.35, and CTC loss. The key innovation was the feature representation: rather than using raw coordinates (x, y, pen), we computed six derivative features per point: differential coordinates $(\Delta x, \Delta y)$, log magnitude $\log(r)$, angle θ, and explicit pen state flags (pen_{up}, pen_{down}). These features capture local stroke geometry and pen movement dynamics, providing a richer representation than raw coordinates alone. The features were normalized using global statistics (mean and standard deviation) computed from the training set, and mild data augmentation—random scaling (0.9–1.1), rotation ($\pm 8^\circ$), and small noise—was applied to training samples. The model was trained from scratch for 170 epochs with batch size 32, learning rate 1×10^{-3}, weight decay 1×10^{-4}, and a ReduceLROnPlateau scheduler with patience 15 and factor 0.5.

Offline Branch

Synthetic Pretraining on OpenHand-Synth The offline model was first trained on the synthetic OpenHand-Synth dataset to help the network learn basic character shapes before working with real handwriting. The network layout uses a residual convolutional encoder followed by a recurrent transcription head. The convolutional backend increases the channel depth from 64 up to 512 feature maps using 3×3 convolutions, batch normalization, and ReLU activations. Max pooling layers reduce the spatial layout while maintaining a wider horizontal axis for the text sequence. This feature map is projected into a 256-dimensional sequence representation, passed through a 3-layer bidirectional LSTM with 256 hidden units, and decoded via a linear classifier optimized under Connectionist Temporal Classification (CTC) loss. Pretraining ran for 25 epochs using the AdamW optimizer, a batch size of 16, and Automatic Mixed Precision (AMP) on CUDA.

Fine-Tuning on Handwriting Recognition Dataset Following pretraining, the network was fine-tuned on the real-world *Handwriting Recognition* images. The images were loaded from the local 1,000-sample training folder and resized to the same uniform 128×512 padded setup. Fine-tuning utilized a lower baseline learning rate to adjust the synthetic weights without destroying the learned character features. Model checkpoints were tracked using the 253-sample validation split, saving the best iteration to `checkpoints/offline/finetuned.pth` based on the lowest validation CER. This allowed the model to adapt to genuine script variations, text slants, and background shadows. An intermediate fine-tuning stage using the IAM dataset was briefly tested during development, but our final pipeline focuses entirely on this setup as it provided the most consistent training artifacts.

Offline Branch Training Methodology

The offline recognition pipeline relies on a structural Sequence-to-Sequence framework optimized to map visual text matrices onto tokenized target indices. Training progresses over a distinct three-stage timeline designed to manage domain transitions:

1. **Synthetic Pretraining:** The network backbone is initialized by optimizing on the 5,000-sample OpenHand-Synth sequence partition. This builds foundational character-level visual priors, weight filters, and structural sequence-to-sequence mappings before exposing the weights to unconstrained human scripts.

2. **Intermediate Fine-Tuning Attempt:** To test target adaptability across complex domain boundaries, the pretrained visual prior is evaluated through an intermediate transfer phase using the offline structures of the IAM handwriting database[15].

3. **Final Fine-Tuning:** Following intermediate transfer trials, the model undergoes targeted local fine-tuning on the real-world Kaggle handwriting-recognition parameters[16] to isolate and stabilize final model convergence.

Multimodal Conditional Feature Fusion

The final system combines the online and offline branches through a confidence-based conditional fusion mechanism. Each branch produces an independent transcription hypothesis, and the fusion layer assigns higher weight to the prediction with the stronger per-sample confidence. This design preserves the strengths of both modalities: the online branch is better at capturing stroke dynamics, while the offline branch is better at using visual shape and texture cues. Conditional fusion was chosen over fixed averaging because it allows the system to adapt to samples where one modality is clearly more reliable than the other.

Interactive User Interface and Implementation

To demonstrate practical use, the project includes an interactive Streamlit-based web interface. The interface provides a drawing canvas for handwritten input and displays the recognition output from the multimodal system in real time. This makes the pipeline usable as a lightweight demo and allows direct inspection of the model predictions without requiring any external software. The UI is not part of the core model evaluation, but it is important for showing how the online and offline branches can be deployed together in a user-facing application.

MinHash and LSH for Document Similarity

To enable efficient similarity search among journal entries, we employed MinHash and Locality Sensitive Hashing (LSH). MinHash approximates the Jaccard similarity between two sets by computing hash signatures of their shingles (character n-grams). For each document, we generated 3-character shingles and computed a 128-permutation MinHash signature. The signatures were indexed using LSH, which hashes similar signatures into the same buckets, allowing efficient approximate nearest neighbor search. This approach reduces the search space from linear to constant time per query, making it suitable for real-time applications with large document collections. We implemented the method using the `datasketch` library and evaluated its performance in terms of query time, recall, and threshold sensitivity.

References

1. Plamondon, R. & Srihari, S. N. On-line and off-line handwriting recognition: A comprehensive survey. *IEEE Transactions on Pattern Analysis & Mach. Intell.* **22**, 63–84 (2000).

2. Puigcerver, J. Offline handwritten text recognition with deep learning. In *Doctoral Consortium of the 14th IAPR International Conference on Document Analysis and Recognition* (2017).

3. Liu, C. *et al.* Col-olhtr: A novel framework for multimodal online handwritten text recognition. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 1–5 (IEEE, 2025).

4. Lodh, A., Chakraborty, R., Palaiahnakote, S. & Pal, U. A transformer based handwriting recognition system jointly using online and offline features. In *Asian Conference on Pattern Recognition*, 250–264 (Springer, 2025).

5. Kunhoth, J., Saleh, M., Al-Maadeed, S. & Akbari, Y. Multimodal ensemble with conditional feature fusion for dysgraphia diagnosis in children from handwriting samples. *Cogn. Comput.* **18**, 10 (2026).

6. Graves, A., Fernández, S., Gomez, F. & Schmidhuber, J. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, 369–376 (2006).

7. Paszke, A. *et al.* PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, vol. 32, 8024–8035 (2019).

8. Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)* (2019).

9. Broder, A. Z. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*, 21–29 (IEEE, 1997).

10. Indyk, P. & Motwani, R. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, 604–613 (1998).

11. Tang, Z., Tang, L., Tang, K. & Tang, R. When similarity digest meets vector management system: A survey on similarity hash function. *arXiv preprint arXiv:2109.08789* (2021).

12. Gervais, P., Deselaers, T., Aksan, E. & Hilliges, O. The DIDI dataset: digital ink diagram data. *Google Cloud Storage* https://console.cloud.google.com/storage/browser/digital_ink_diagram_data (2020). Paper available at https://arxiv.org/abs/2002.09303.

13. Liwicki, M. & Bunke, H. IAM-OnDB – an on-line english sentence database acquired from handwritten text on a whiteboard. *FKI Document Analysis Group* https://fki.tic.heia-fr.ch/databases/download-the-iam-on-line-handwriting-database (2006). Published in: Proceedings of the 8th International Conference on Document Analysis and Recognition (ICDAR 2006).

14. Adeyanju, I., Omidiora, E., Fenwa, O., Babalola, O. & Durodola, O. ISGL Online and Offline Character Recognition Dataset. Mendeley Data https://data.mendeley.com/datasets/n7kmd7t7yx/1, DOI: 10.17632/n7kmd7t7yx.1 (2019).

15. Marti, U.-V. & Bunke, H. The IAM-database: an English sentence database for offline handwriting recognition. *Int. J. on Document Analysis Recognit.* **5**, 39–46 (2002).

16. landlord. Handwriting recognition dataset. Kaggle Datasets https://www.kaggle.com/datasets/landlord/handwriting-recognition (2020). Accessed: 2026-07-09.

17. Lhoest, Q. *et al.* Datasets: A community library for natural language processing and computer vision. *arXiv preprint arXiv:2109.02846* (2021).

Author contributions statement

Dž.K. conceived all the necessary work on the offline branch, including data processing, and K.T. on the online branch. The authors analysed the results of their respective model branches. The Streamlit application was reviewed by all authors. All authors reviewed the manuscript.

Additional information

Accession codes

All data generated or analysed during this study are included in this article. All code used in this experiment is available on the MuDi-HTR Github repository: `https://github.com/kanitafro/MuDi-HTR`

Competing interests

The authors declare no competing interests.